Prompt Forge Coding Standards

1. Naming Conventions

1.1 Variables

Local variables: camelCase starting with a lowercase letter.

Example: promptCount, userSession.

Constants: UPPER_SNAKE_CASE.

Example: MAX_LOGIN_ATTEMPTS.

Must match Checkstyle regex: ^[a-z][a-zA-Z0-9]*$ (variables) or ^[A-Z][A-Z0-9]*(_[A-Z0-9]+)*$ (constants).

1.2 Parameters

Function/method parameters must be camelCase.

Example: userId, maxRetries.

1.3 Member Variables

Class member variables must be camelCase.

Example: userRepository, promptService.

1.4 Functions / Methods

Names must be camelCase, start with lowercase letter.

Example: handleSubmit() , fetchUserData().

Must describe intent clearly (no abbreviations like procUsr).

1.5 Classes, Interfaces, Enums

PascalCase, starting with uppercase.

Example: UserProfile, PromptService.

Must match Checkstyle regex: ^[A-Z][a-zA-Z0-9]*$.

1.6 Packages

Lowercase only, words separated by dots.

Example: com.promptforge.authentication.

1.7 Files

Frontend React components: PascalCase (UserProfilePage.tsx).

Backend utility/config files:PascalCase .

Example: UserController.java

---

2. Code Structure

2.1 Indentation and Formatting

- Frontend: 2-space indentation
- Backend: 2-space indentation
- No tabs
- Line length:
    - 100 characters (frontend)
    - 100 characters (backend)
    - Exceptions: imports(packages) and URLs(http://, https://)

2.2 Comments

- Only comment complex or non-obvious logic
- Use // for single-line, /* */ for multi-line

2.3 Single Statement per Line

Correct:

```
const user = getUser();

const prompts = getPrompts();
```

Incorrect:

```
const user = getUser(); const prompts = getPrompts();
```

---

3. Error Handling

3.1 Try-Catch Blocks

- Wrap risky operations (I/O, DB, APIs) in try-catch
- Keep blocks focused and minimal

3.2 Logging

- Use **Winston** (frontend) and **SLF4J** (backend)
- Log essential debug/error info only
- Never log sensitive data (e.g., passwords, tokens)

3.3 Error Propagation

- Re-throw with context when necessary
- Use custom error types where applicable

3.4 User-Facing Errors

- Show user-friendly, actionable messages
- Avoid exposing stack traces or sensitive logic
- Avoid using technical jargon

3.5 Error Codes

Use standard HTTP status codes:

- 400 Bad Request

- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Internal Server Error

3.6 Validation

- Validate all incoming data at service or controller level
- Return clear, structured error responses on failure
- Return structured JSON errors

---

4. Testing Standards

4.1 Types of Tests

- **Unit Testing**
  - Tools: Cypress (frontend), JUnit (backend)
- **Integration Testing**
  - Tools: Spring Boot Test, MSW, React Testing Library
- **End-to-End (E2E) Testing**
  - Tools: Cypress
  - Should be executed via CI for full-stack validation

4.2 Code Coverage

- Minimum 70% coverage for all critical modules
- Tools: --coverage with Cypress using Codecov, JaCoCo with Spring Boot

---

5. Git and Branching Standards

5.1 Git Flow Model

- main – stable production-ready branch
- dev – active development branch
- feature/<name> – new features
- hotfix/<name> – urgent patches
- test/<name> – test configurations
- docs/<name> – documentation

5.2 Branch Naming Conventions

- Use kebab-case
  Examples:

    - feature/add-user-auth
    - bugfix/fix-prompt-validation
    - docs/update-readme

5.3 Review Process

- PRs must:
    - Target develop (dev)
    - Include clear description and task references
    - Pass all tests and linters (workflows)
    - Be reviewed and approved by at least one other dev

5.4 Linting

- Frontend: ESLint + Prettier
- Backend: Checkstyle + SpotBugs
- All linters enforced via CI

---

6. Project Structure

```
Prompt-Forge/
├── backend/
│   └── project/
│       └── src/main/java/com/fiveOps/promptforge/
│           ├── analytics/
│           ├── authentication/
│           │   ├── controller/
│           │   ├── dto/
│           │   └── service/
│           ├── dashboard/
│           ├── databaseConfig/
│           ├── prompts/
│           └── resources/
```

```
│       └── application.properties
├── frontend/
│   └── src/
│       ├── components/
│       ├── pages/
│       ├── services/
│       └── App.tsx
├── public/
├── package.json
├── tsconfig.json
└── README.md
```

---

7. API Design

- Use RESTful design principles
- Endpoint structure: /api/users
- Validate input and return structured JSON responses
- Handle exceptions centrally (@ControllerAdvice)
- Document endpoints using Swagger
- Implement rate limiting for sensitive or high-traffic routes

---

8. Configuration & Tooling

- Store secrets in .env (never commit secrets)
- Use dotenv-safe to enforce variable presence
- Use Docker Compose for full-stack local environments

---

9. Documentation

- Maintain README.md, CONTRIBUTING.md
- Document APIs (Swagger), Java (Javadoc), TS/JS (TSDoc)

- Include usage examples in modules and service layers
